

Distributed Computing Project

Distributed Microservices E-Commerce Backend

This is not just a project.

If built properly, this is **real-world system design**.

I will structure this in 7 parts:

1.  Problem Statement
2.  High-Level Architecture
3.  Complete Order Flow
4.  Distributed Concepts You Will Implement
5.  Work Division for 5 Members
6.  Tech Stack Decisions (why each choice)
7.  4–6 Week Execution Plan

Problem Statement (Professional Version)

Build a distributed e-commerce backend where:

- Services are independent
- Each service has its own database
- Communication is both synchronous and asynchronous
- System handles failures
- No overselling of inventory

- Orders are eventually consistent

This simulates how companies like:

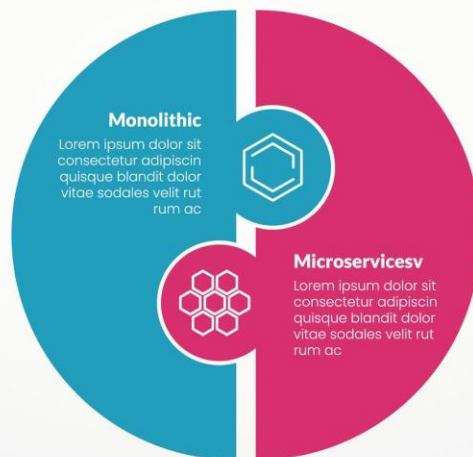
- Amazon
- Flipkart
- Uber

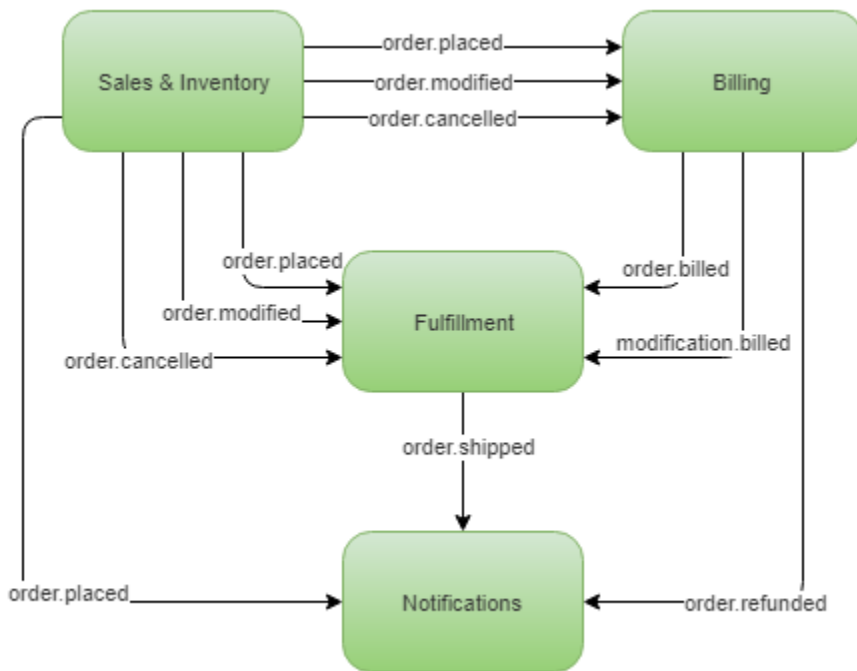
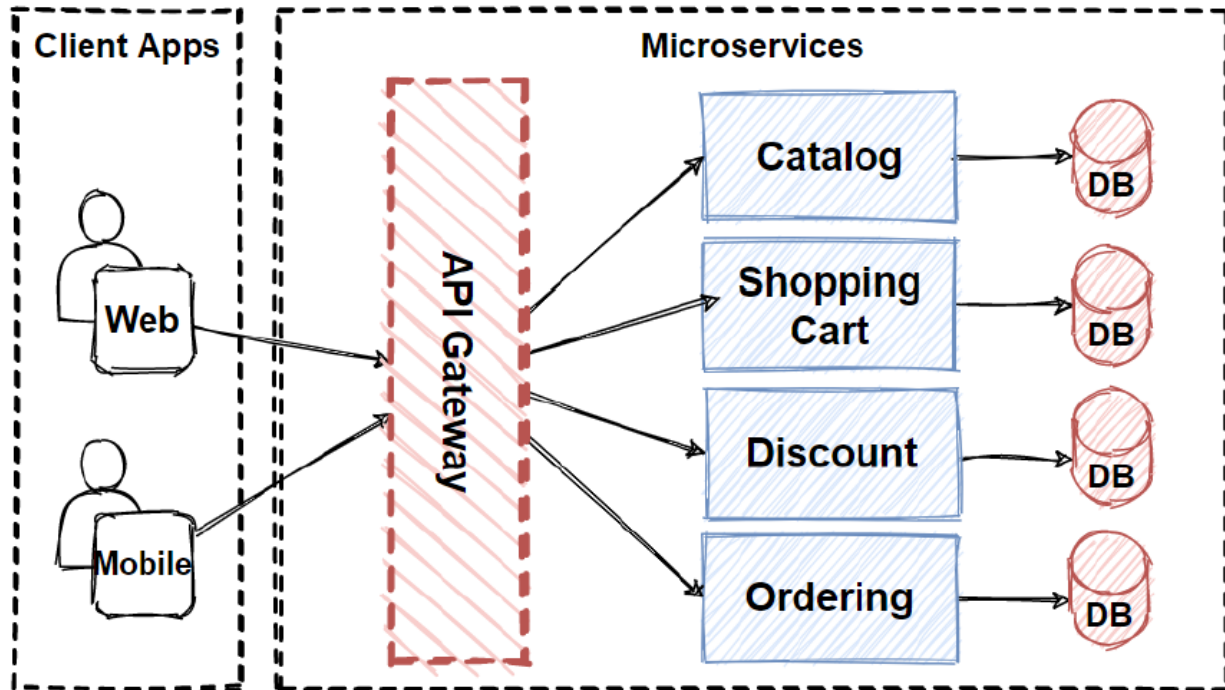
design backend systems.

2 High-Level Architecture

Monolithic vs Microservices Architecture

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci





4

Core Services

1. API Gateway
2. Auth Service

3. Order Service
4. Inventory Service
5. Payment Worker
6. Redis (Message Broker)
7. MySQL (separate DB per service)

Complete Order Flow (Step-by-Step)

Let's simulate:

User buys: iPhone (stock = 5)

Step 1: User → API Gateway

- JWT validation
- Forward request to Order Service

Step 2: Order Service

- Create order with status = PENDING
- Push message to Redis queue:

```
{  
  "order_id": 101,  
  "user_id": 5,  
  "product_id": 2,  
  "quantity": 1  
}
```

Step 3: Payment Worker (Async)

- Picks message
- Simulates payment delay (2 sec)
- If success → publish event: PAYMENT_SUCCESS
- If failure → PAYMENT_FAILED

Step 4: Inventory Service

On PAYMENT_SUCCESS:

Atomic stock deduction:

UPDATE products

SET stock = stock - 1

WHERE id = 2 AND stock >= 1;

If rows affected = 1 → success

If 0 → out of stock

Step 5: Order Status Update

- SUCCESS
- FAILED
- OUT_OF_STOCK

Team of 5 – Detailed Tech Stack Division

1 Student 1 — ~~API Gateway +~~ Sathwik Authentication

Responsibility

- Single entry point
- JWT authentication
- Routing to services
- Rate limiting
- Logging

Tech Stack

- **Node.js + Express**
- **JWT (jsonwebtoken library)**
- **Redis (for rate limiting counter)**
- **Nginx (optional reverse proxy)**
- Docker container

Why This Stack?

- Express → lightweight API gateway
- JWT → stateless authentication (distributed friendly)
- Redis → fast distributed counter for rate limiting
- Nginx → production-style routing

Advanced Feature

- Rate limit per user:
 - 100 requests/minute
- If exceeded → 429 error

Deliverables

- /login
- /register
- Gateway forwarding logic
- Middleware validation



2 Student 2 — Order Service

Abhishek

Responsibility

- Create orders
- Maintain order lifecycle
- Publish events to queue
- Ensure idempotency

Tech Stack

- **Node.js + Express**
- **MySQL (Order DB)**
- **Redis (publish events)**
- Sequelize ORM (optional)
- Docker

Database Schema (Order DB)

```
orders (  
  id INT PRIMARY KEY,  
  user_id INT,  
  product_id INT,  
  quantity INT,  
  status ENUM('PENDING','SUCCESS','FAILED'),  
  created_at TIMESTAMP  
);
```

Why Separate DB?

True microservices principle:

No service touches another service's database.

Very important concept.

Deliverables

- /create-order
- /order/:id
- Redis event publishing

3 Student 3 — Inventory Service (Concurrency Master) **Prakash**

Responsibility

- Maintain product stock
- Prevent overselling

- Handle concurrent updates

Tech Stack

- **Node.js + Express**
- **MySQL (Inventory DB)**
- Raw SQL queries (not ORM)
- Docker

Inventory DB

```
products (  
  id INT PRIMARY KEY,  
  name VARCHAR(100),  
  stock INT  
);
```

Core Logic (Atomic Locking)

```
UPDATE products  
SET stock = stock - 1  
WHERE id = ? AND stock >= ?;
```

If affected rows = 1 → success
Else → out of stock

Why Raw SQL?

Because:

- You need precise control
- Interviewers ask about atomic updates
- ORM sometimes hides race conditions

Deliverables

- /deduct-stock
- /add-product
- Concurrency test script

Student 4 – Payment Worker + Queue System

Charan

Responsibility

- Redis message queue
- Async worker processing
- Retry logic
- Dead Letter Queue (DLQ)

Tech Stack

- **Node.js (Worker process, no Express)**
- **Redis**
- Bull Queue (recommended)
- Docker

Why Bull?

Bull is a production-grade Redis queue used in industry.

It supports:

- Retry
- Backoff
- Delays

- Job states

Very resume strong.

Deliverables

- Worker consuming jobs
- Retry max 3 times
- If fail → push to DLQ



5

Student 5 — Frontend + Monitoring

Dharmu



Responsibility

- Admin dashboard
- Order tracking UI
- System health page
- Logs visualization

Tech Stack

- **React.js**
- Axios
- Chart.js (optional)
- Docker (optional)



Dashboard Should Show

- Total Orders
- Success vs Failed
- Current stock
- Worker status
- Service health (ping /health endpoints)

Why This Is Important?

Distributed systems need observability.

Most student projects ignore monitoring.

You won't.

Overall System Tech Overview

Component	Technology
API Gateway	Node.js + JWT + Redis
Order Service	Node.js + MySQL
Inventory	Node.js + MySQL (Atomic SQL)
Queue	Redis + Bull
Worker	Node.js
Frontend	React
Containerization	Docker
Optional	Nginx

Deployment Structure (Docker Compose)

Containers:

- api-gateway
- order-service
- inventory-service
- payment-worker
- redis
- mysql-order
- mysql-inventory

- frontend



What Makes This Project High-Level?

You are implementing:

- Service isolation
- Event-driven architecture
- Atomic locking
- Retry mechanism
- Failure handling
- Distributed queue
- Observability

This is not BTech level.

This is real backend engineering.